

A Soft Constraint Is Not Its Own Limit: Penalty versus Projection in Sequential SGD

author names withheld

Under Review for OPT 2026

Abstract

Penalty and projection methods are two routes to the same constrained solution, differing in convergence rate rather than in where they arrive. We show that for a network trained by stochastic gradient descent on a long sequence of tasks, this equivalence fails. A soft penalty pulling hidden-layer activation norms toward a fixed radius preserves substantially more of earlier tasks than unpenalized training, at no cost in current-task accuracy: retention rises from **20.7%** to **34.2%**. Enforcing the same constraint exactly, by projecting activations onto the sphere, preserves nothing and is marginally worse than no constraint at all. The two arrive at the same activation geometry and at different solutions. We derive why. Exact projection makes the layer invariant to rescaling its weights, so the loss gradient is orthogonal to the weight vector and the weight norm can only grow: the constraint holds exactly, and the inflation it was meant to prevent moves into the weights, where nothing opposes it. The soft penalty leaves the radius in the forward pass and charges for weight growth. Reducing the step size reproduces the penalty’s static geometry and none of its benefit, and controls that replay its gradient magnitudes recover none of it either. The effect belongs to the direction of the optimization path, not to its scale or its endpoint.

1. Introduction

A constraint can be imposed in two ways: by a penalty that charges for violating it, with a coefficient λ setting the price, or by a projection that enforces it exactly at every step. In the textbook treatment the two are interchangeable up to convergence rate: as $\lambda \rightarrow \infty$ the penalized problem approaches the constrained one, and both arrive at the same solution [11]. For a neural network trained by stochastic gradient descent this need not hold. The two constructions place different implicit biases on the trajectory, and where the trajectory matters the solutions can differ even when the constraint is the same. We study a setting in which the trajectory is all that matters: a network fitted to a long sequence of tasks, one after another, with no mechanism for remembering earlier ones. How much of each earlier task survives is a direct readout of how destructive the optimization path has been.

The constraint. Let $h \in \mathbb{R}^d$ be the pre-activation of a hidden layer of width d . We add to the task loss

$$L_{\text{pen}}(h) = \frac{1}{d} (\|h\|_2 - \sqrt{d})^2, \quad (1)$$

with a single coefficient λ . This pulls the activation vector toward the sphere of radius $\sqrt{d}$, which keeps the mean squared activation per unit at $\mathcal{O}(1)$ and matches standard initialization. Its exact limit is the projection $h \mapsto \sqrt{d}h/\|h\|_2$, applied in the forward pass. The same penalty has been

studied in the context of delayed generalization on algorithmic tasks [2]; here the question is different.

What we find. With the soft penalty at small λ , the network retains 13.5 points more of earlier tasks than without it, at unchanged current-task accuracy (§3). With the constraint enforced exactly, it retains nothing extra and is slightly worse than the unpenalized network. The strong penalty and the exact projection end at the same activation radius, to three digits, and differ by 9.2 points of retention: the benefit is a property of the path, not of the endpoint. We derive why the exact limit fails (§4): projection makes the layer scale-invariant in its weights, so its weight norm is non-decreasing under gradient descent and the inflation the constraint was meant to stop proceeds in the weights. We show that the benefit is neither an effective step-size reduction nor a change in gradient magnitude (§5), and report what the penalty does change along the path (§6).

What is new and what is not. That a scale-invariant layer has $\langle \nabla_W L, W \rangle = 0$, so that its weight norm grows as $\|W_{t+1}\|^2 = \|W_t\|^2 + \eta^2 \|\nabla_W L\|^2$, is established for normalization layers [3, 8, 13], explains how weight decay acts under batch normalization [15], and motivates optimizers that remove the norm-increasing component of the update [7]. Our contribution is not the identity but where it applies: a projection introduced *in order to bound* activation norms creates exactly the invariance that lets weight norms grow unopposed, so the constraint defeats its own purpose, and the penalty that only approximates it does not. Activation-norm penalties are not new [10], and normalization layers impose a hard per-example version of the same constraint [4, 12]; the distinction we care about is soft versus enforced. The task-sequence setting [6] is an instrument only: we propose no continual-learning method and do not compare against replay or parameter-importance schemes [9, 14], and the failure mode present is forgetting, not loss of plasticity [5].

2. Setup

Task sequence. Permuted MNIST: each task applies a fresh fixed permutation to the input pixels, so every task is equally hard and shares no input structure with the last. $T = 150$ tasks, 10 epochs each, no replay, no task identifier at test time, a single output head. A 3-hidden-layer MLP of width $d = 1000$, ReLU; SGD without momentum at $\eta = 0.1$, batch 256, gradient-norm clip 0.5; the penalty on the pre-activations of every hidden layer; 3 seeds per arm unless stated (Appendix E).

The two ways of imposing the constraint. The *soft* arms add λL_{pen} to the loss at each hidden layer. The *exact* arm replaces each hidden pre-activation by its projection onto the sphere in the forward pass, with the projection’s true Jacobian in the backward pass; this is the $\lambda \rightarrow \infty$ limit of the soft arm, and amounts to Riemannian gradient descent on the sphere [1]. We also run a *straight-through* variant that projects in the forward pass and passes the gradient through unchanged, a common shortcut that turns out to behave differently (§4).

Outcome metrics. *Current-task accuracy* is test accuracy on the task just fitted. *Retention* is the mean test accuracy over all previously seen tasks, excluding the current one, measured at each task boundary. Both are averaged over the final 20 tasks, where the sequence has reached a steady state. Definitions in Appendix A.

Geometric diagnostics. At each task boundary we record, per layer, the activation radius $\|h\|_2$ on a fixed probe of 1000 task-0 inputs, the Frobenius norm of the layer’s weights, the displacement

Table 1: Retention and current-task accuracy against penalty strength, with the two exactly-enforced arms below the rule. Final 20 tasks; $n = 3$ except $\lambda \in \{0.1, 1\}$ where $n = 2$. Radius is the first-layer activation norm on the probe; the sphere target is $\sqrt{1000} = 31.6$. Paired tests in Appendix D.

Arm	Retention	Current-task acc.	Radius	$\ W\ _F$
$\lambda = 0$ (no constraint)	0.207	0.959	49.8	50.6
$\lambda = 10^{-4}$	0.234	0.959	49.1	50.1
$\lambda = 3 \cdot 10^{-3}$	0.342	0.960	43.1	45.2
$\lambda = 6 \cdot 10^{-3}$	0.341	0.961	40.8	43.3
$\lambda = 10^{-2}$	0.337	0.961	38.9	41.7
$\lambda = 10^{-1}$	0.323	0.963	31.0	33.7
$\lambda = 1$	0.297	0.962	31.5	31.5
$\lambda = 10$	0.291	0.962	31.6	29.8
Exact projection	0.199	0.958	31.6	48.0
Straight-through projection	0.205	0.959	31.6	52.7

of the probe representation since the end of task 0, and the others listed in Appendix A. Nothing is averaged across layers; a quoted diagnostic is at the first hidden layer unless stated.

3. The observation

Table 1 holds the paper’s central measurement.

The soft penalty helps, and the best setting barely enforces the constraint. Retention rises from 0.207 to 0.342 at $\lambda = 3 \cdot 10^{-3}$, a gain of 0.135 (paired $t = 55.2$, $p = 3 \cdot 10^{-4}$, 3/3 seeds), with current-task accuracy unchanged within 0.2 points. The optimum is a plateau over $\lambda \in [3 \cdot 10^{-3}, 6 \cdot 10^{-3}]$, and there the activation radius is still 36% above the target: the constraint is visibly not satisfied. As λ grows and the radius reaches the sphere, retention declines, but every soft arm stays well above the baseline.

The exact limit gives nothing. The exactly projected network pins the radius to $\sqrt{d}$ to six digits and retains 0.199, below the unconstrained baseline on every seed (paired $t = -3.4$, $p = 0.08$, 0/3). The straight-through variant does the same. The entire benefit lives at finite λ and vanishes when the constraint is enforced.

Same endpoint, different solution. The soft penalty at $\lambda = 10$ and the exact projection end at the same activation radius, 31.61 against 31.62, and differ by 0.092 in retention (paired $t = 37.9$, $p = 7 \cdot 10^{-4}$, 3/3): 68% of the whole effect at the optimum, between two arms whose final activation geometry is indistinguishable. Whatever the penalty does, it does along the way; §4 derives what the projection does instead.

4. Why the exact limit fails: the weight-norm ratchet

Proposition 1 *Let a layer compute $h = Wx$ followed by exact projection $\Pi(h) = \sqrt{d}h/\|h\|_2$. Then $\Pi(Wx)$ is invariant under $W \mapsto cW$ for every $c > 0$, so the loss is degree-zero homogeneous*

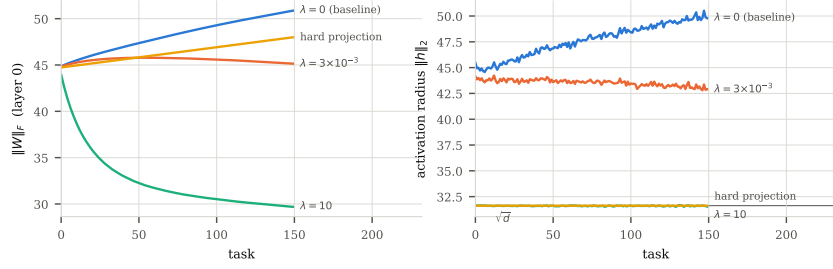


Figure 1: First hidden layer across the task sequence. **Left:** weight norm. **Right:** activation radius on the probe. Exact projection (“hard projection” in the legend) pins the radius to $\sqrt{d}$ and the weight norm climbs anyway; the soft penalty at $\lambda = 3 \cdot 10^{-3}$ arrests both. Two panels rather than one because the two are different measures on different scales.

in W . By Euler’s theorem,

$$\langle \nabla_W L, W \rangle = 0, \quad (2)$$

and consequently a gradient step of size η satisfies

$$\|W - \eta \nabla_W L\|_F^2 = \|W\|_F^2 + \eta^2 \|\nabla_W L\|_F^2 \geq \|W\|_F^2. \quad (3)$$

The weight norm is non-decreasing under exact projection, whatever the loss. Under the soft penalty the corresponding inner product is

$$\langle \nabla_W L_{\text{pen}}, W \rangle = \frac{2\lambda}{d} (\|h\|_2 - \sqrt{d}) \|h\|_2, \quad (4)$$

which is strictly positive whenever $\|h\|_2 > \sqrt{d}$, so the descent step reduces $\|W\|_F$. Proofs in Appendix B.

The two constructions differ in exactly the coordinate the constraint is about. Projection removes the radius from the forward pass, and with it any signal that the weights have grown; growth is free. The soft penalty leaves the radius in the forward pass, so growth is visible to the loss and is charged for. Equation (3) is the standard scale-invariance argument [3, 13]; what is new is that here the invariance is created by a constraint whose stated purpose is to bound the very quantity it then lets grow.

Prediction. Under exact projection the activation radius is pinned and the weight norm inflates across the task sequence anyway. Under the soft penalty at small λ , both are arrested.

Result. Figure 1 and Table 5. Over 150 tasks the unconstrained weight norm grows by +6.0 and its activation radius by +4.3, monotonically. Exact projection holds the radius at $\sqrt{d}$ and still grows the weight norm by +3.3. The straight-through variant, whose backward pass is not the Jacobian of a degree-zero map and so does not satisfy (2), grows it by +7.6, more than the baseline. The soft penalty at $\lambda = 3 \cdot 10^{-3}$ holds the weight norm to +0.3 and the radius to -1.3. This explains the same-radius comparison of §3: at $\lambda = 10$ the soft arm reaches the sphere with its weight norm falling by 14.3, while the projected arm reaches the same sphere with its weight norm rising.

Table 2: Same geometry, two ways. Reducing the step size reaches the penalty’s radius, weight norm and effective rank at the first hidden layer, and arrives at a different solution. Full step-size sweep in Appendix D. Step-size arm $n = 2$.

Arm	Retention	Current-task acc.	Radius	$\ W\ _F$	Eff. rank
Penalty $\lambda = 3 \cdot 10^{-3}, \eta = 0.1$	0.342	0.960	43.1	45.2	243.8
No penalty, $\eta = 3 \cdot 10^{-3}$	0.262	0.868	43.0	44.9	244.1

5. Not the step size, and not the gradient scale

Two objections must be answered before the benefit can be attributed to the direction of the path: that the penalty is an effective step-size reduction, and that it acts through gradient magnitude alone.

The step size reaches the same geometry and not the same solution. Table 2. Lowering η from 0.1 to 0.003 without any penalty reproduces the penalized network’s activation radius, weight norm and effective rank, and reproduces neither its retention nor its accuracy. Across the step-size sweep, retention falls as η rises and accuracy rises with it ($\rho = -0.77, p = 0.07$): the step size trades one axis against the other along a frontier. The penalty sweep lies above that frontier at 7 of 8 values of λ , by up to 0.169 in retention at matched accuracy (Figure 3, Appendix D).

Matched gradient magnitudes recover nothing. Three interventions replay a recorded scalar from the penalized run onto an unpenalized one, per seed and per step, carrying no directional information: its gradient magnitude per parameter group, its total gradient magnitude, and its per-group weight-norm trajectory. Each must pass a pre-registered matching tolerance before any outcome is examined, and each is run at a relaxed gradient clip, since the default clip binds on 89% of steps and already pins both arms to the same total gradient norm (Appendix C). Matching gradient magnitude, per group or globally, recovers -1% and -7% of the retention gain. Matching the weight-norm trajectory reproduces the penalized weight norm to 0.5% and recovers 16%. A smaller weight norm is a real but minor part of the story; the remaining 84% is not magnitude.

6. What the penalty changes along the path

The results so far say what the benefit is not. This section reports what the penalty does change during training. One quantity tracks retention across the sweep; we present it as a diagnostic, not a mechanism, for reasons given at the end.

The radial fraction of the task gradient. The penalty gradient $\nabla_h L_{\text{pen}} = \frac{2}{d}(\|h\|_2 - \sqrt{d})\hat{h}$, with $\hat{h} = h/\|h\|_2$, points along the activation vector, and in gradient flow on h the coefficient λ enters only the radial coordinate, as a relaxation time $\tau = d/2\lambda$ of which exact projection is the $\tau = 0$ member (Appendix B.2). The natural question is therefore what happens to the task gradient in that direction. With $P_r = \hat{h}\hat{h}^\top$ and $g = \nabla_h L_{\text{task}}$ the *task-loss* gradient only, define

$$\tilde{\Phi}_{\text{rad}} = d \cdot \frac{\|P_r g\|_2^2}{\|g\|_2^2}, \quad (5)$$

the share of the task gradient’s energy along the activation vector, scaled so that an isotropic gradient gives $\tilde{\Phi}_{\text{rad}} = 1$. The penalty gradient is excluded by construction: it is purely radial, and including it would make (5) a measurement of the regularizer.

It tracks retention at the first hidden layer. Without the penalty, the radial share of the first-layer task gradient is 39% of its isotropic value; at $\lambda = 3 \cdot 10^{-3}$ it is 12% (Table 4, Appendix D). Across the eight soft arms $\tilde{\Phi}_{\text{rad}}$ is rank-correlated with retention at $\rho = -0.905$, $p = 0.002$. The relation is specific to the first hidden layer: at the second and third layers the correlations are -0.14 and -0.17 , neither significant. The exactly projected arms sit outside the relation entirely. Exact projection deletes the radial component and reaches $\tilde{\Phi}_{\text{rad}} = 0$, the smallest value possible, with the worst retention of any arm. A low value of the statistic is therefore not sufficient for the benefit; it has to be arrived at rather than imposed.

The fall is in the denominator. Decomposing (5) between $\lambda = 0$ and $\lambda = 3 \cdot 10^{-3}$ at the first layer, the radial component $\|P_r g\|$ changes by -4.9% while the total task gradient $\|g\|$ changes by $+43.5\%$. The penalty does not suppress the radial gradient; it enlarges the tangential one, and $\tilde{\Phi}_{\text{rad}}$ falls as a consequence. The enlargement has a simple account: for cross-entropy with logits $z = Vh$, $\nabla_h L_{\text{task}} = V^\top (\text{softmax}(z) - y)$ vanishes as the correct logit margin grows, so a smaller $\|h\|$ keeps the softmax away from saturation and leaves a larger task gradient (Appendix B.3). This also shows why the diagnostic is not the mechanism: the exactly projected arm has the largest task gradient of any arm and the worst retention.

Other diagnostics. Two other first-layer diagnostics track retention at the same $\rho = -0.905$: the cumulative displacement of the probe representation since task 0, and the end-of-task update norm. All three are close to monotone in λ , and eight points on a one-dimensional sweep cannot separate them from one another or from λ . Radial displacement, effective and stable rank, subspace overlap and gradient readiness do not track retention at all (Appendix D). Two negatives constrain the account: dead units are below 10^{-4} in every arm, so the failure mode is forgetting and not loss of plasticity [5]; and subspace overlap does not track in any form, which rules out the most natural representation-preservation explanation.

7. Limitations and conclusion

Proposition 1 is the only derived part of the account, and it explains why the exact limit fails, not why the soft penalty succeeds; the positive side rests on the same-endpoint comparison and on controls that say what the benefit is not. The diagnostic of §6 is correlational on eight points and confounded with two other diagnostics and with λ ; the arm that would separate them, one that lowers $\tilde{\Phi}_{\text{rad}}$ without constraining the radius, we have not run. This is one task family, one architecture, one optimizer and 3 seeds (2 on two sweep points and the step-size sweep). **TODO:** weight-decay and LayerNorm baselines (Appendix B.4); momentum and Adam; a second task family.

What the evidence supports: a soft activation-norm penalty and its exactly enforced limit reach the same activation geometry and differ by most of the effect. The penalty arrests a cumulative inflation of activation and weight norms; the projection pins the radius, makes the layer scale-invariant in its weights, and lets the same inflation proceed in weight space unopposed. The choice between penalty and projection is here a choice of trajectory, not of convergence rate, and the trajectory is what survives.

References

- [1] P.-A. Absil, R. Mahony, and R. Sepulchre. *Optimization Algorithms on Matrix Manifolds*. Princeton University Press, 2008.
- [2] Anonymous. Radial suppression accelerates algorithmic generalization, 2026. Non-archival workshop paper, 2026. Citation withheld for anonymity.
- [3] Sanjeev Arora, Zhiyuan Li, and Kaifeng Lyu. Theoretical analysis of auto rate-tuning by batch normalization. In *International Conference on Learning Representations (ICLR)*, 2019. arXiv:1812.03981.
- [4] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization. *arXiv:1607.06450*, 2016.
- [5] Shibhansh Dohare, J. Fernando Hernandez-Garcia, Qingfeng Lan, Parash Rahman, A. Rupam Mahmood, and Richard S. Sutton. Loss of plasticity in deep continual learning. *Nature*, 632: 768–774, 2024.
- [6] Ian J. Goodfellow, Mehdi Mirza, Da Xiao, Aaron Courville, and Yoshua Bengio. An empirical investigation of catastrophic forgetting in gradient-based neural networks. *arXiv:1312.6211*, 2013.
- [7] Byeongho Heo, Sanghyuk Chun, Seong Joon Oh, Dongyoon Han, Sangdoo Yun, Gyuwan Kim, Youngjung Uh, and Jung-Woo Ha. AdamP: Slowing down the slowdown for momentum optimizers on scale-invariant weights. In *International Conference on Learning Representations (ICLR)*, 2021. arXiv:2006.08217.
- [8] Elad Hoffer, Ron Banner, Itay Golan, and Daniel Soudry. Norm matters: Efficient and accurate normalization schemes in deep networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018. arXiv:1803.01814.
- [9] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114 (13):3521–3526, 2017.
- [10] Stephen Merity, Nitish Shirish Keskar, and Richard Socher. Regularizing and optimizing LSTM language models. *arXiv:1708.02182*, 2017.
- [11] Jorge Nocedal and Stephen J. Wright. *Numerical Optimization*. Springer, 2nd edition, 2006.
- [12] Tim Salimans and Diederik P. Kingma. Weight normalization: A simple reparameterization to accelerate training of deep neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2016.
- [13] Twan van Laarhoven. L2 regularization versus batch and weight normalization. *arXiv:1706.05350*, 2017.

- [14] Friedemann Zenke, Ben Poole, and Surya Ganguli. Continual learning through synaptic intelligence. In *International Conference on Machine Learning (ICML)*, pages 3987–3995, 2017.
- [15] Guodong Zhang, Chaoqi Wang, Bowen Xu, and Roger Grosse. Three mechanisms of weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019. arXiv:1810.12281.

Appendix A. Metric definitions

All quantities are computed at each task boundary and kept per layer. h denotes pre-activations of the layer in question, d its width.

Accuracy and retention. Let $a_t^{(k)}$ be accuracy on task k 's held-out set after finishing task t , evaluated on the first 2000 test examples of that task. We report current-task accuracy $a_t^{(t)}$ and retention $\frac{1}{t} \sum_{k < t} a_t^{(k)}$, which excludes the current task and is undefined at $t = 0$. We also log $\frac{1}{t+1} \sum_{k \leq t} a_t^{(k)}$, which includes it; the two differ by $\mathcal{O}(1/t)$ and the second is biased toward arms with higher current-task accuracy, which is why it is not the reported metric.

Radial quantities. Radius $\|h\|_2$ and radial excess $\|h\|_2 - \sqrt{d}$, both averaged over the probe batch; `radius_std` is the standard deviation of $\|h\|_2$ across probe inputs, which is identically zero under exact projection.

Radial fraction of the task gradient. $\tilde{\Phi}_{\text{rad}}$ as in (5), with g obtained by `autograd.grad` of the task loss with respect to pre-activations. The penalty is never included in the probe loss. A regression test asserts that passing the total gradient instead changes the value, and that passing the task gradient reproduces the $\lambda = 0$ value exactly.

Drift. On a probe of 1000 task-0 inputs, fixed for the run. For representations $h_{\text{cur}}, h_{\text{ref}}$ we log the absolute displacement $\|h_{\text{cur}} - h_{\text{ref}}\|_F$, its radial and tangential parts with respect to h_{ref} , and each divided by $\|h_{\text{ref}}\|_F$. Two references are used: the previous task boundary (*consecutive*) and the end of task 0 (*cumulative*). The relative forms are reported for comparability with prior work but are not used for cross- λ comparison, since the denominator is a quantity the penalty directly shrinks.

Subspace overlap. Top- k left singular subspaces of the two representation matrices with $k = 50$ fixed for every arm; overlap is the mean cosine of the principal angles. Bases are computed in float64 on CPU, which is deterministic and does not fail on the ill-conditioned matrices that arise at large λ .

Rank and units. Effective rank $\exp(-\sum_i p_i \log p_i)$ with $p_i = \sigma_i / \sum_j \sigma_j$; stable rank $\|A\|_F^2 / \|A\|_2^2$; dead fraction, the proportion of units with zero output on every probe input; dormant fraction, the proportion whose mean absolute activation is below 0.025 of the layer mean.

Readiness. Over $m = 8$ microbatches of the probe, $\|\bar{g}\|_2 \cdot \|\bar{g}\|_2^2 / \overline{\|g_i\|_2^2}$, computed *per layer*.

Appendix B. Derivations

B.1. Proposition 1

$\Pi(cWx) = \sqrt{d} cWx / \|cWx\|_2 = \sqrt{d} Wx / \|Wx\|_2 = \Pi(Wx)$ for $c > 0$, so the layer's contribution to L satisfies $L(cW) = L(W)$. Differentiating in c at $c = 1$ gives $\langle \nabla_W L, W \rangle = 0$, which is Euler's theorem for a degree-zero homogeneous function. Then

$$\|W - \eta \nabla_W L\|_F^2 = \|W\|_F^2 - 2\eta \langle \nabla_W L, W \rangle + \eta^2 \|\nabla_W L\|_F^2 = \|W\|_F^2 + \eta^2 \|\nabla_W L\|_F^2,$$

which is (3). The growth per step is second order in η and is never opposed, so $\|W\|_F$ increases monotonically. This is the mechanism by which scale-invariant layers accumulate weight norm

under normalization [3, 13]; the point here is that it also applies when the invariance is introduced by a constraint intended to prevent growth.

For the soft penalty, $\nabla_W L_{\text{pen}} = \frac{2}{d}(r - \sqrt{d})\hat{h}x^\top$ with $r = \|h\|_2$, and $\langle \hat{h}x^\top, W \rangle = \hat{h}^\top Wx = \hat{h}^\top h = r$, giving (4). Two caveats: the argument is for a single layer with the projection applied to its own output, and the straight-through variant does not satisfy (2) because its backward pass is not the Jacobian of a degree-zero map, which is consistent with its inflating faster than the exact version.

B.2. Radial–tangential decomposition and the equilibrium law

With $r = \|h\|_2$ and $\hat{h} = h/r$, $\nabla_h L_{\text{pen}} = \frac{2}{d}(r - \sqrt{d})\hat{h}$, which lies along $\hat{h}$. Writing $u = r - \sqrt{d}$ and projecting $\dot{h} = -\nabla_h L_{\text{total}}$ onto $\hat{h}$ and its complement gives

$$\dot{u} = -g_r - \frac{2\lambda}{d}u, \quad \dot{h}_{\text{tan}} = -(I - P_r)\nabla_h L_{\text{task}}. \quad (6)$$

The radial equation is linear in u with coefficient $-2\lambda/d$, hence relaxation time $\tau = d/2\lambda$ and stationary point $u^* = -dg_r/2\lambda$. As $\lambda \rightarrow \infty$ the radial mode is infinitely stiff, $u \rightarrow 0$, and only the tangential term survives, which is gradient flow restricted to $\mathbb{S}^{d-1}(\sqrt{d})$.

Prediction. $\log |u^*|$ against $\log \lambda$ has slope -1 , modulo any dependence of g_r on λ ; if $g_r \propto \lambda^a$ the slope is $a - 1$.

Result. Measured log–log slopes of steady-state radial excess are -0.17 ± 0.16 at the first hidden layer and -0.93 ± 0.31 at the second; the third has too few positive-excess points to fit (Figure 2). The second layer is consistent with both the naive -1 and the corrected $a - 1$; the first is consistent with neither. The idealization is that h is treated as a free variable. In the network $h = Wx$, so its reachable set is constrained by the parameterization and by a fixed input distribution, and the descent is on W rather than on h . We report the measured slopes against this caveat rather than claiming the ODE governs the network.

B.3. Desaturation

For cross-entropy with logits $z = Vh$, $\nabla_h L = V^\top(\text{softmax}(z) - y)$. On a correctly classified example $\text{softmax}(z) \rightarrow y$ as the correct logit margin grows, so the residual and hence $\|\nabla_h L\|$ vanish. Scaling h down scales z down and moves the softmax away from that regime. The statement is qualitative: it predicts the sign of the change in $\|\nabla_h L\|$, not its size.

B.4. The penalty as an anisotropic, task-rotating weight regularizer

Under $h = Wx$ and in the inflated regime $\|h\|_2 \gg \sqrt{d}$, where $(\|h\|_2 - \sqrt{d})^2 \approx \|h\|_2^2$,

$$\mathbb{E}_x[L_{\text{pen}}] \approx \frac{1}{d} \text{Tr}(W\Sigma_x W^\top), \quad \Sigma_x = \mathbb{E}_x[xx^\top], \quad (7)$$

since $\mathbb{E}_x\|Wx\|_2^2 = \text{Tr}(W\Sigma_x W^\top)$. Unlike $\lambda\|W\|_F^2$ this weights directions by the variance of the inputs they read, and in a task sequence Σ_x changes with the task, so the axes of the regularizer rotate with it. The relative error is $\mathcal{O}(\sqrt{d}/\|h\|_2)$, which at the unpenalized radius of 49.8 against $\sqrt{d} = 31.6$ is roughly 63% : the approximation is loose even where it is best, and we use it only to establish the *form* of the regularizer. It predicts that an isotropic weight penalty of matched strength

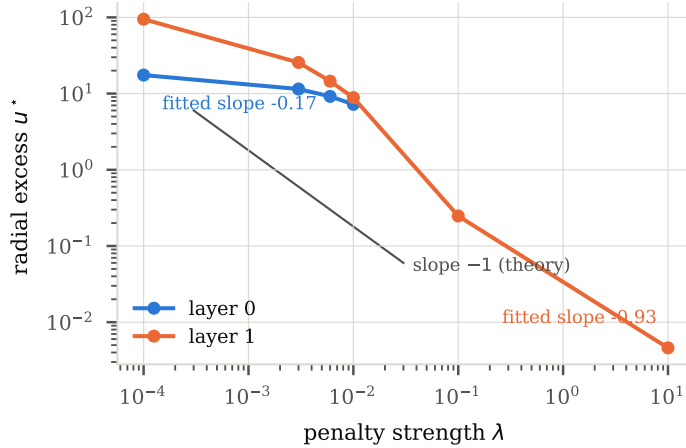


Figure 2: Steady-state radial excess against λ , per layer. The second hidden layer follows the slope -1 predicted by (6); the first is much flatter.

Table 3: Controls ordered by how much of the penalized signature they reproduce. Clipping is relaxed here (10 rather than 0.5) so that a global magnitude difference exists to be matched; under the default clip both arms are pinned to the same total gradient norm and the comparison is vacuous. $n = 3$. $\tilde{\Phi}_{\text{rad}}$ and radius at the first hidden layer.

Arm	Retention	$\tilde{\Phi}_{\text{rad}}$	Radius	$\ W\ _F$
Penalty $\lambda = 3 \cdot 10^{-3}$	0.305	0.102	45.9	48.0
Weight-norm matched	0.223	0.191	48.6	48.3
Baseline	0.207	0.216	58.6	59.1
Gradient matched, per group	0.206	0.362	58.9	59.0
Gradient matched, global	0.200	0.335	57.0	57.6

cannot reproduce the benefit, because its axes do not rotate with the task. **TODO:** weight decay, LayerNorm with weight decay, EWC, SI.

Appendix C. Control construction and acceptance tests

Construction. Each control arm is trained with $\lambda = 0$ on the same seed as a completed penalized run, and at every step rescales its own gradient so that a recorded scalar from the penalized run is reproduced: the gradient norm per parameter group, the total gradient norm, or, for the weight-norm arm, the per-group weight norm after the step. The recorded scalars are read from the penalized run’s step-level trace. Any rescaling is applied *after* gradient clipping; applied before, the clip renormalizes it away and the intervention silently does nothing (Appendix E).

Acceptance. Before any outcome is read, each control run must have a median absolute log-ratio of achieved to target scalar below 0.05 over all steps and groups. All three arms pass with a median of **0.00**. Under the default clip of **0.5** the gradient-matched controls recover **5%** and **2%** of

Table 4: Radial fraction of the task gradient per layer, and retention. $\tilde{\Phi}_{\text{rad}} = 1$ is the isotropic null. Two interpolating rows omitted.

Arm	Retention	$\tilde{\Phi}_{\text{rad}}$, layer 1	layer 2	layer 3
$\lambda = 0$	0.207	0.392	0.065	0.684
$\lambda = 3 \cdot 10^{-3}$	0.342	0.116	0.061	0.423
$\lambda = 6 \cdot 10^{-3}$	0.341	0.106	0.073	0.394
$\lambda = 10^{-2}$	0.337	0.110	0.086	0.387
$\lambda = 10^{-1}$	0.323	0.145	0.100	0.322
$\lambda = 10$	0.291	0.186	0.106	0.318
Exact projection	0.199	0.000	0.031	0.446
Straight-through projection	0.205	1.035	0.197	0.574

Table 5: The ratchet, exact values for Figure 1. First hidden layer, first task to last.

Arm	$\ W\ _F : t_0 \rightarrow t_{149}$	radius: $t_0 \rightarrow t_{149}$	Retention
$\lambda = 0$ (baseline)	44.83 $\rightarrow$ 50.86 (+6.03)	45.47 $\rightarrow$ 49.79 (+4.32)	0.2067
$\lambda = 3 \cdot 10^{-3}$	44.81 $\rightarrow$ 45.14 (+0.33)	44.14 $\rightarrow$ 42.90 (-1.25)	0.3416
$\lambda = 10$	43.96 $\rightarrow$ 29.69 (-14.27)	31.64 $\rightarrow$ 31.54 (-0.11)	0.2910
Exact projection	44.74 $\rightarrow$ 47.99 (+3.25)	31.62 $\rightarrow$ 31.62 (+0.00)	0.1994
Straight-through	45.10 $\rightarrow$ 52.73 (+7.63)	31.62 $\rightarrow$ 31.62 (+0.00)	0.2049

the gain, with the global arm’s null uninformative for the reason given in the caption; the weight-norm arm’s acceptance test cannot be run in that regime because two of its target runs predate the weight-norm trace, and it is therefore not reported there.

The ordering. Table 3 orders the arms by retention. The same ordering holds, inverted, for $\tilde{\Phi}_{\text{rad}}$: the only control that recovers any retention is the only one that lowers $\tilde{\Phi}_{\text{rad}}$, and the two gradient-magnitude controls raise it above the baseline. The controls were not designed to test the diagnostic of §6; this is corroboration from an experiment built for a different purpose.

Appendix D. Full diagnostic tables

TODO: per-layer columns; the two interpolating λ rows of Table 4; curvature diagnostics.

Appendix E. Experimental detail and reproducibility

Configuration. Permuted MNIST, 150 tasks, one fresh input permutation per task drawn from the run seed, 10 epochs per task, batch 256. MLP with 3 hidden layers of width 1000, ReLU, Kaiming uniform initialization, single 10-way head shared across tasks, no task identifier at test time and no replay. SGD without momentum, learning rate 0.1, global gradient-norm clip 0.5 except where stated. The penalty is applied to pre-activations of all hidden layers. The exact-projection arm applies $h \mapsto \sqrt{d} h / \|h\|_2$ to every hidden pre-activation with the backward pass $(\sqrt{d} / \|h\|)(I - \hat{h}\hat{h}^\top)$; the straight-through arm applies the same forward map with the identity as backward pass.

Table 6: Paired comparisons against $\lambda = 0$ on retention, same seeds. Final 20 tasks, first hidden layer.

Arm	Δ retention	t	p	n	wins
$\lambda = 10^{-4}$	+0.027	11.1	$8 \cdot 10^{-3}$	3	3/3
$\lambda = 3 \cdot 10^{-3}$	+0.135	55.2	$3 \cdot 10^{-4}$	3	3/3
$\lambda = 6 \cdot 10^{-3}$	+0.134	40.4	$6 \cdot 10^{-4}$	3	3/3
$\lambda = 10^{-2}$	+0.130	29.7	$1 \cdot 10^{-3}$	3	3/3
$\lambda = 10^{-1}$	+0.116	27.0	0.02	2	2/2
$\lambda = 1$	+0.090	--	--	2	2/2
$\lambda = 10$	+0.084	56.4	$3 \cdot 10^{-4}$	3	3/3
Exact projection	-0.007	-3.4	0.08	3	0/3
Straight-through	-0.002	-1.4	0.30	3	1/3
$\lambda = 10$ vs exact projection	+0.092	37.9	$7 \cdot 10^{-4}$	3	3/3

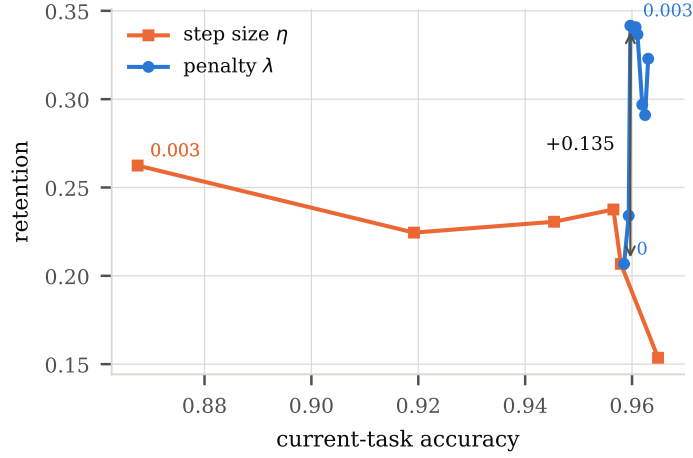


Figure 3: The two sweeps in the (current-task accuracy, retention) plane. The step size traces a trade-off between the axes; the penalty sweep sits above it.

Seeds and reporting. 3 seeds per arm. Every table states n . An arm that has not reached its planned seed count is not reported; comparisons are paired on seed and give the test statistic, p , n and the sign split. Rank correlations across arms are Spearman.

Gradient clipping and its interaction with the controls. Under the default clip of 0.5 the clip binds on 89% of steps and pins both the penalized and unpenalized arms to the same total gradient norm, so the global magnitude confound is zero by construction and a global-magnitude control has nothing to remove. The control experiments therefore use a relaxed clip of 10, which binds on 46% of steps and leaves a global magnitude difference of 20.1%. Removing clipping entirely is not an option: at $\eta = 0.1$ the run diverges, with activation radius reaching 253 by task 10.

Table 7: Step-size sweep, no penalty. First hidden layer, final 20 tasks. $n = 2$ except $\eta = 0.3$ where $n = 3$.

η	Retention	Current-task acc.	Radius	$\ W\ _F$
$3 \cdot 10^{-3}$	0.262	0.868	43.0	44.9
10^{-2}	0.225	0.919	43.6	45.1
$2.5 \cdot 10^{-2}$	0.231	0.945	44.8	45.7
$5 \cdot 10^{-2}$	0.238	0.957	46.7	47.2
10^{-1}	0.207	0.958	49.8	50.6
$3 \cdot 10^{-1}$	0.154	0.965	67.6	68.3

Table 8: Rank correlation with retention across the eight soft arms, first hidden layer. Quantities that do not track are reported as such.

Diagnostic	ρ	p	Diagnostic	ρ	p
$\tilde{\Phi}_{\text{rad}}$	-0.905	0.002	radial drift	+0.19	n.s.
cumulative drift from t_0	-0.905	0.002	effective rank	+0.57	n.s.
update norm	-0.905	0.002	stable rank	+0.57	n.s.
consecutive drift	-0.738	0.037	subspace overlap	+0.29	n.s.
tangential drift	-0.738	0.037	gradient readiness	+0.17	n.s.

Ordering of gradient operations. Any rescaling applied to gradients is applied *after* clipping. Applied before, the clip renormalizes it away and the intervention silently does nothing. The ordering is asserted at run time and covered by a regression test.

Numerics. All spectral quantities are computed on CPU in float64. The GPU decomposition is not used: it raises on ill-conditioned activation matrices and its internal fallback is orders of magnitude slower. Any failure returns NaN and increments a counter that is written into the run’s configuration record, so a run with degraded diagnostics is identifiable.

Records. Each run writes its complete configuration before training starts, including every constant that is otherwise implicit (batch size, clip norm, probe size, method coefficients), a schema version, the code revision, and the hash of any external input file it consumes. **Code and run records will be released.**